hashlock.

Security Audit

Tapir Money (DeFi)

June 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

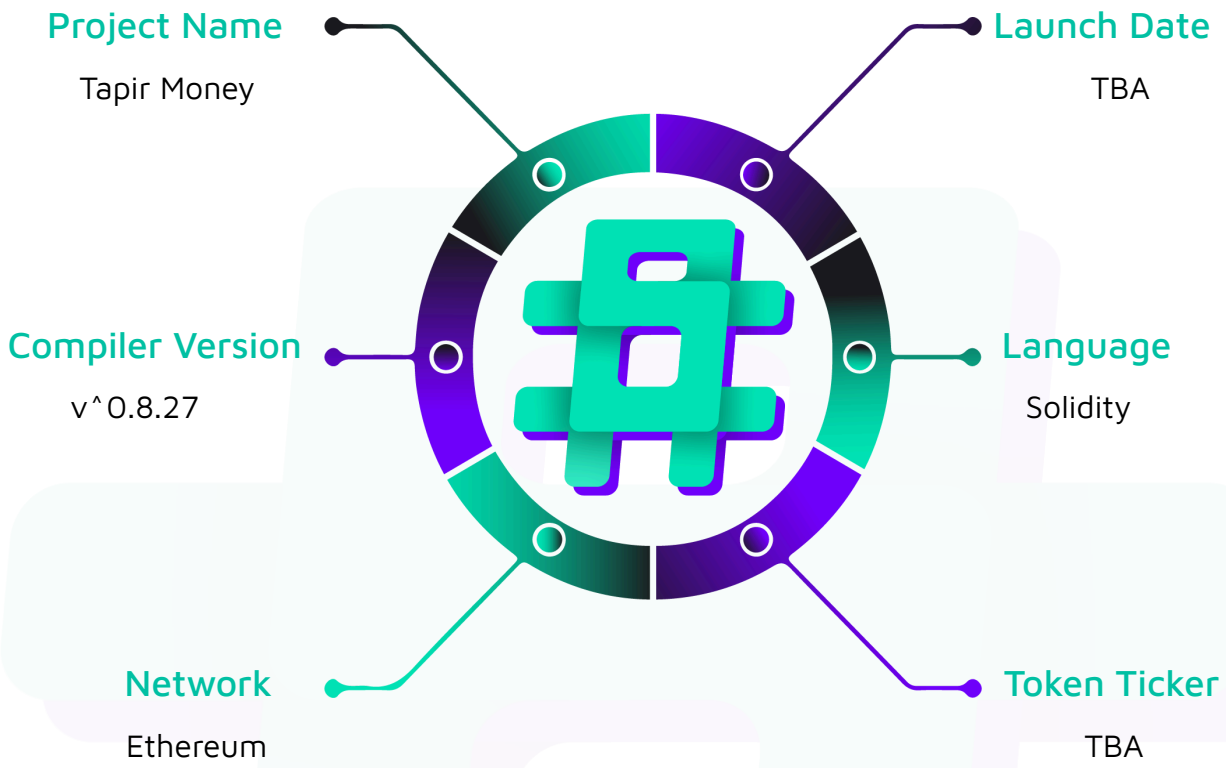
The Tapir Money team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

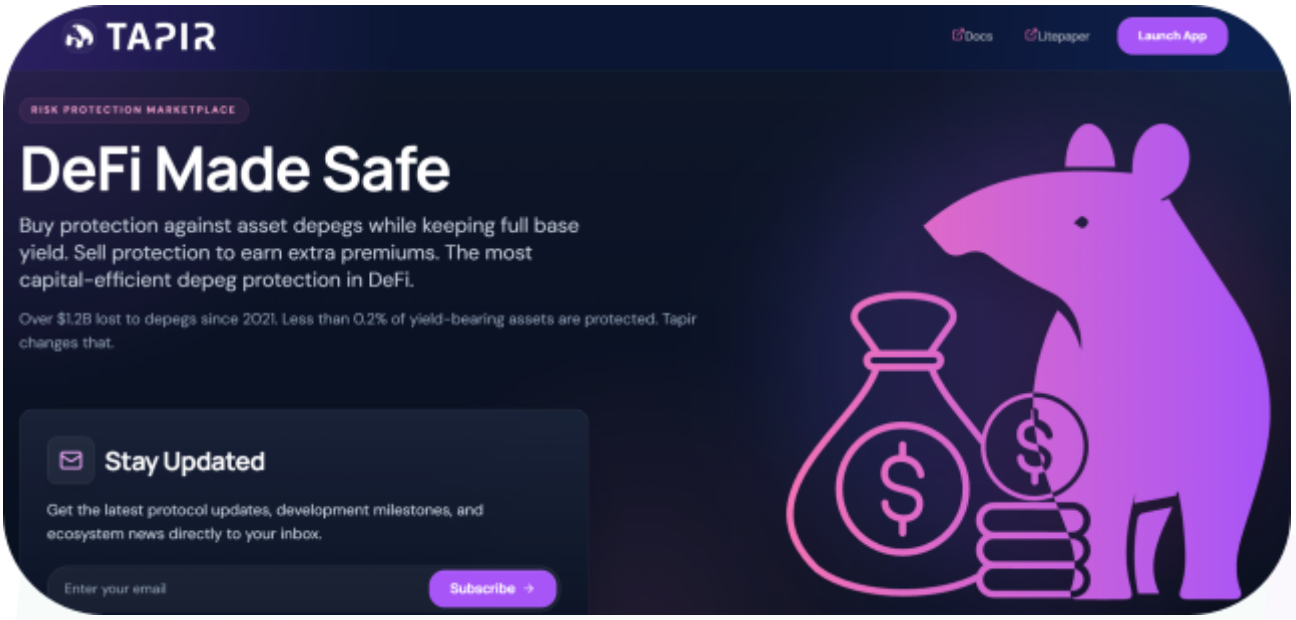
Tapir is a decentralized finance (DeFi) protocol focused on depeg risk management. It operates as a marketplace where users can either purchase protection against stablecoin or liquid staking token depegs or earn additional yield by selling that protection. Unlike traditional DeFi insurance models that require idle collateral, Tapir keeps capital productive by splitting yield-bearing assets into protection-focused and yield-boosted positions, allowing users to customize their risk exposure while continuing to earn underlying returns. The protocol aims to improve capital efficiency and make DeFi risk hedging more accessible for yield-seeking investors.

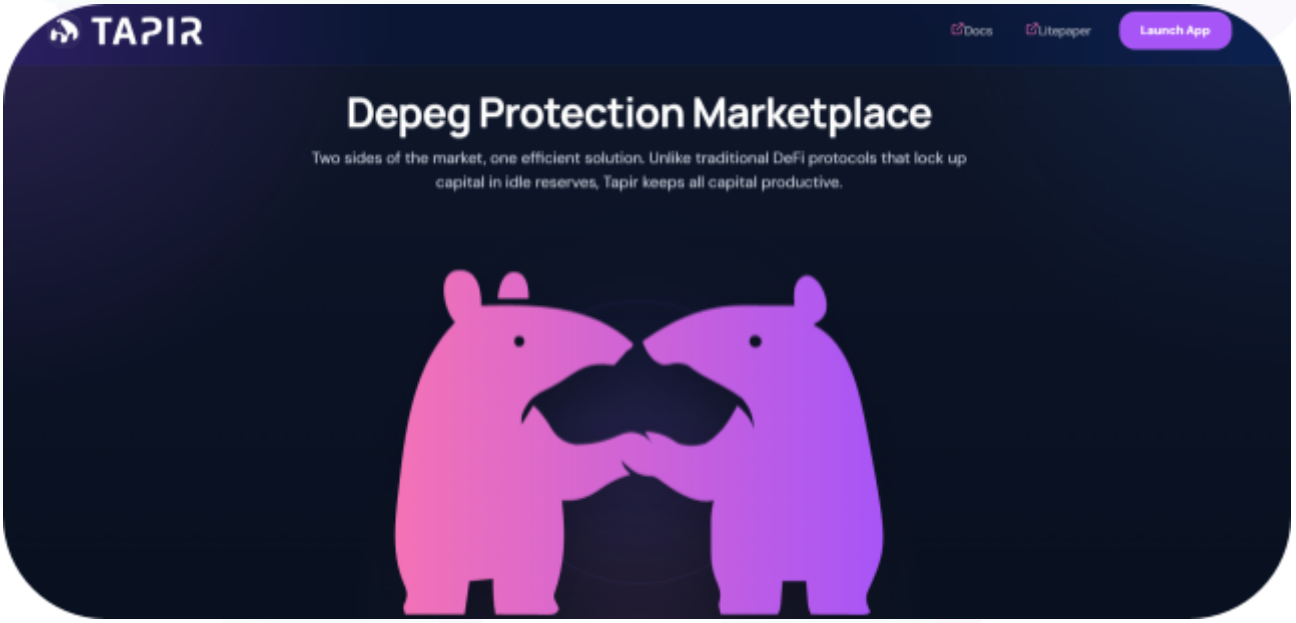
Project Name: Tapir Money
Project Type: Defi
Compiler Version: ^0.8.27
Website: https://tapir.money/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Tapir Money project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Tapir Money Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**June, 2026**
Contract 1	contracts/DepegFactory.sol
Contract 2	contracts/DepegPool.sol
Contract 3	contracts/TapirOracle.sol
Contract 4	contracts/TapirPtrwOracle.sol
Contract 5	contracts/TapirRouter.sol
Contract 6	contracts/TapirVrpOracle.sol
Contract 7	contracts/libraries/TimeDecay.sol
Audited GitHub Commit Hash	e951edc34939d8191d65c5ef29c2fc2e4ac5a462
Fix Review GitHub Commit Hash	9fc06de7ad6531771ab245e01828459d9b0b00a7

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerability

11 Low severity vulnerabilities

20 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

TapirOracle.sol
- Allows the admin to:
  - Configure price feed sources via setSources
  - Update oracle parameters (minValidSources, checkpointSpacing, maxStaleness) via setConfig
  - Pause and unpause checkpoint creation
- Allows the operator to:
  - Record fresh prices from each individual feed (Chainlink, API3, RedStone Classic, Tellor)
  - Commit a new price checkpoint aggregating valid, non-stale sources
  - Write the finalised HWM and closing price to the connected DepegPool
	Contract achieves this functionality.

TapirVrpOracle.sol
- Allows the operator to:
  - Record checkpoint alongside a VRP ring-buffer snapshot in a single call
  - Resolve VRP by sampling the vault's live previewRedeem at settlement time
  - Write VRP-adjusted HWM and closing price to the DepegPool
- Allows the admin to:
  - Manually supply a PRV value if the auto resolveVrp path returns zero
	Contract achieves this functionality.

(manualBackupResolveVrp)	

TapirPtrwOracle.sol
- Allows the operator to:
  - Resolve PTRW by redeeming PT tokens held in the oracle via the Pendle router, recording ERV and ARV
  - Write PTRW-adjusted HWM and closing price to the DepegPool
- Allows the admin to:
  - Manually supply ERV and ARV if the auto resolvePtrw path fails (manualBackupResolvePtrw)
	Contract achieves this functionality.

DepegPool.sol
- Allows anyone to:
  - Split base tokens into equal DP and YB tranches while the pool is ACTIVE
  - Unsplit DP and YB back into base tokens while the pool is ACTIVE
  - Trigger COOLDOWN state if the oracle price breaches the depeg threshold
  - Redeem DP tokens for their pro-rata payout once the pool reaches RESOLUTION
  - Redeem YB tokens for their pro-rata payout once the pool reaches RESOLUTION
- Allows the oracle to:
  - Write finalised HWM and closing price data during COOLDOWN, advancing the pool toward RESOLUTION
	Contract achieves this functionality.

- Allows the admin to: - Sweep accumulated split fees to the treasury address	

DepegFactory.sol
- Allows the admin to:
  - Deploy and register new DepegPool instances with validated oracle, treasury, and configuration parameters
  - Support cross-chain deployment mode where the oracle address may be an EOA rather than a contract
	Contract achieves this functionality.

TapirRouter.sol
- Allows anyone to:
  - Split base tokens and immediately purchase a specific tranche (DP or YB) in a single transaction via splitAndBuy
  - Sell a tranche back and unsplit to base in a single transaction via sellAndUnsplit
  - Perform a direct token swap between supported assets via swapExactIn
	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Tapir Money project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Tapir Money project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] DepegPool - ProposeOracleChange unusable for cross-chain pools

Severity rationale

Final verdict, Fixed (historically Medium). Original scan severity, High (later Medium). Status against the current source, Fixed.

Description

`DepegPool.proposeOracleChange` now wraps the code/`cfg()` validation in `if (xDomainMessengerL2 == address(0))` (`DepegPool.sol:257`), mirroring the factory's cross-chain carve-out. An L2 pool can now rotate to a new L1 oracle. No further action. Add a regression test that proposes an L1 address with no L2 bytecode under `xChainMode`.

Status

Resolved

Low

[L-01] VRP bootstrap can normalize an early vault loss into the HWM

Severity rationale

Final verdict, Low. Original scan severity, Medium. Status against the current source, Open.

Description

With daily VRP previews `[100, 50, 50]`, the HWM drops from the constructor baseline of 100 to `min(100,50,50) = 50` at the third daily observation, so a 50% loss compares against a 50 HWM and reports no depeg.

Three corrections to the original write-up. First, `initialVrpPreview` is not an asymmetric normalization anchor; it divides out of the depeg ratio, since `writePriceData` scales HWM and closing by the same `/initialVrpPreview`. Second, the `< 3 daily → maxObserved` fallback is unreachable inside `writePriceData`: the inherited `_highWatermarkPrice()` reverts `InsufficientCheckpointsForHWM` below three daily price points, and the VRP daily count is always at least the price daily count. Third, the behaviour is not VRP-specific: the base `TapirOracle` shows the identical collapse for a price series `[2, 1, 1]`, and the `max(min(triplet))` semantics match the design intent that the high-water mark reflect a price held for at least three days.

The actual gap is the absence of a warm-up gate between oracle deployment and pool activation: the deployment flow creates the oracle and pool in one step, so no healthy history is required before users can enter. Three healthy daily observations before the loss preserve the HWM and the depeg is detected. Impact is value misallocation, but confined to the first ~3 days of oracle life, so likelihood is Low; nothing decays the HWM afterward. Reject the proposed fix `max(maxOfMins, initialVrpPreview)`: it makes a single 1-day observation a permanent floor, defeating the exact 3-day rule it belongs to. Correct fix: require at least three distinct healthy daily observations before pool activation.

Status

Acknowledged

[L-02] Resolution accepts arbitrarily stale checkpoints

Severity rationale

Final verdict, Low (enabling half of AI-19). Original scan severity, Medium. Status against the current source, Open.

Description

`writePriceData` never reads `lastCheckpointTs`; `_getRecentCheckpointCount()` floors to `MIN_CHECKPOINTS_FOR_MEDIAN = 5` even when zero checkpoints sit inside `closingPriceLookbackPeriod`, and `_medianOfLastEligibleCheckpoints` silently consumes out-of-window entries. Settlement then succeeds on year-old data: `hwm` and `closing` both hold at the stale peg while the true market has moved, so a real 50% depeg reports `poolHasDepegged = false`. `DepegPool.resolvePriceDepeg` enforces only a minimum price age, no maximum.

As a standalone staleness bug it needs a checkpointing outage far beyond "a few blocks" (assumption 2), which drops likelihood. It matters mainly as the enabling half of AI-19: the same floor-of-5 reach-outside-lookback is what lets a single day dominate the closing median. Merge it with AI-19 and fix both with one change: count eligible days, not raw checkpoints, and enforce an explicit maximum fallback age.

Status

Acknowledged

[L-03] TapirOracle - Chainlink circuit-breaker floors unhandled; 2-source mode pins to a stale peg

Severity rationale

Final verdict, Low. Original scan severity, Medium (detail Info). Status against the current source, Open.

Description

`_recordAggregatorV3Price` (`TapirOracle.sol:234-237`) validates only `answer <= 0`, `updatedAt == 0`, future timestamp, and monotonic timestamp; no `minAnswer`/`maxAnswer` and no round-completeness check. A feed clamped at its `minAnswer` keeps writing heartbeat rounds, so `updatedAt` advances and it never goes stale; the 2-valid-source "closer to previous" branch (`:297-318`) then selects it on every checkpoint through a 0.95→0.50 crash, reporting a 50% depeg as 0.00%. With three active sources the median tracks the crash exactly, so the defect is specific to the two-source branch.

Why Low, not Medium. Two forces pull it down. First, a `minAnswer` clamp is a feed failure, and assumption 3 promises the majority of feeds are robust; with only two active sources there is no majority, so the assumption is arguably violated rather than protective. That argument keeps the mechanic valid. Second, and decisive for severity: the two-source branch is not reachable in the shipped configuration, which runs a single active source (`minValidSources = 1`), and with three or more sources the median absorbs one clamped feed. That reachability gap pins it at Low. Recommendation: where feeds expose `minAnswer`/`maxAnswer`, reject clamped values, and enforce three or more active sources on production pools (the team's own mitigation). Note that the single-source shipped configuration contradicts that mitigation and should be revisited.

Status

Acknowledged

[L-04] TapirPtrwOracle - ResolvePtrw pays the entire PT redemption proceeds to the caller as a "tip"

Severity rationale

Final verdict, Low (value routing) plus Informational (natspec). Original scan severity, Info. Status against the current source, Open.

Description

`resolvePtrw()` transfers the oracle's entire `pendleBase` balance (`TapirPtrwOracle.sol:121-125`), the full redemption proceeds plus any pre-existing balance, to `msg.sender`. On 1,000 units of proceeds and a 25-unit prior donation, the operator receives all 1,025; none is routed to the pool or a treasury. The natspec is self-contradictory across four lines (`:74` "Safe to call by anyone", `:76`/`:77` operator-only).

The recipient is the `OPERATOR_ROLE`, trusted under assumption 2, so this is value routing by design rather than theft, and the loss scales with the deposited witness amount, which the design recommends keeping small. Route proceeds to the treasury and pay a bounded tip; fix the natspec.

Status

Resolved

[L-05] Five raw checkpoints can collapse into one daily closing observation

Severity rationale

Final verdict, Low (the closest call; the reasoning for why it does not reach Medium is below). Original scan severity, Low. Status against the current source, Open.

Description

The design defines the closing price as the median of the five most recent daily values in a per-day secondary array. The implementation counts raw checkpoints in the lookback window (floor of 5) and aggregates to daily afterward, then medians whatever days result. It fails two ways:

- Too few days. With the shipped VRP configuration (a 12-hour lookback and 2-hour checkpoint spacing), 20 healthy days at 2.0 plus five design-sanctioned final-day samples during a 1-day dip yield closing 1.0, so the pool settles `dpValue 20000 / ybValue 0` instead of `10000/10000`.
- Too many days. With a 20-day lookback the code medians every day in the window, not the five most recent. A plain 10% APY monotonic yield asset yields `hwm 1.004451 / closing 1.002485`, a 0.196% drop that crosses the 0.1% gate, so `poolHasDepegged = true` and `principalDepeggedValue = 9980`, contradicting the design's own statement that the closing price is never below the high-water mark.

Three forces hold it below Medium:

1. Source of truth. Under a strict reading, the engagement brief is the governing specification, and it is silent on closing-price methodology; the five-daily-median formula lives in the separate oracle design note. The code's own header comment describes the implemented behaviour, so there is no self-contradiction. That weakens the spec-deviation basis.
2. Minimal-loss cap on the realistic magnitude. The config-independent branch (the monotonic-yield asset) is a logic defect but produces only ~0.2% misallocation, squarely the rounding / fee-discrepancy cap, so Low even with infinite repeatability.
3. The large-magnitude branch needs an unusual event. The `20000 / 0` outcome

requires a ~50% single-day move landing at maturity, sampled at high cadence, so Low likelihood; a realistic few-hundred-bp blip yields a correspondingly small misallocation.

By the matrix, High impact × Low likelihood is Medium, but the minimal-loss cap on the realistic case and the brief-not-design-note source-of-truth reading pull the defensible verdict to Low. It is the one finding whose trigger is a natural market event rather than a privileged actor or a config choice, so it is the finding to escalate if the design note is treated as binding. Recommendation (fixes AI-8 and AI-19 together): build the eligible daily-price array first, require at least five distinct eligible days, take the median of the latest five daily representatives, and do not silently reach outside the configured lookback; enforce an explicit maximum fallback age instead.

Status

Acknowledged

[L-06] TapirVrpOracle#writePriceData - VRP xChain lock

Severity rationale

Final verdict, Low. The code expresses no such guard; add `if (cfg_.xChainMode) revert` in the VRP constructor.

Description

`TapirVrpOracle.writePriceData` reads `IDepegPool(depegPool).MAX_PRICE()` directly (`:151`); in `xChainMode` the L2 pool has no code on L1, so every write reverts and the pool locks in COOLDOWN forever. Neither the constructor nor `setConfig` rejects the configuration, and the factory supports xChain pools.

Impact

Impact ceiling if the trust model is broken. High (fund lock)

Gating clause under the agreed assumptions. Deploy-time parameter (missing validation); team has it on record as "must not be used in xChainMode"

Recommendation

The code expresses no such guard; add `if (cfg_.xChainMode) revert` in the VRP constructor

Status

Resolved

[L-07] TapirVrpOracle#checkpoint - VRP checkpoint DoS on vault pause

Severity rationale

Final verdict, Low.

Description

`TapirVrpOracle.checkpoint()` calls `previewRedeem` with no non-VRP fallback, so an honest vault pause or upgrade stops base-feed checkpoints too, while `writePriceData` still settles on stale data.

Impact

Impact ceiling if the trust model is broken. Medium (temporary DoS)

Gating clause under the agreed assumptions. Vault-reliability assumption plus admin recovery (`manualBackupResolveVrp`, 7-day oracle swap)

Status

Acknowledged

[L-08] Resolve→write yield drift

Severity rationale

Final verdict, Low. Size `errorTolerance` above the expected resolve→write drift, or re-quote PRV at write.

Description

`vrpPrv` is snapshotted at `resolveVrp` (`:92`) versus a live HWM at `writePriceData` (`:137`); honest yield between the two reads as a depeg. It crosses the 0.1% gate at 6 days at 10% APY, 4 days at 20%, and 150 bp at 30 days, on a vault that never lost value. It needs at least one intervening checkpoint, and `errorTolerance` suppresses it.

Impact

Impact ceiling if the trust model is broken. Medium (minor loss)

Gating clause under the agreed assumptions. Minimal-loss cap; honest-cadence assumption

Recommendation

Size `errorTolerance` above the expected resolve→write drift, or re-quote PRV at write

Status

Acknowledged

[L-09] PTRW manual-resolve latch

Severity rationale

Final verdict, Low. Add a reset and a bounded PT rescue.

Description

`forceManualResolve` (`:78`) has no reset; a single zero `netTokenOut` (standard tokens, no exotic behaviour) bricks `resolvePtrw()` permanently, and with no sweep function the held PT is stranded.

Impact

Impact ceiling if the trust model is broken. Low

Gating clause under the agreed assumptions. Standard-token path, no malice needed

Recommendation

Add a reset and a bounded PT rescue

Status

Resolved

[L-10] TapirRouter - Router balance accounting

Severity rationale

Final verdict, Low.

Description

`sellAndUnsplit` accounts by `balanceOf(this)` (`TapirRouter.sol:140-163`) and `swapExactIn` trusts a caller-supplied venue's return value; any balance stranded in the router is claimable by any caller.

Impact

Impact ceiling if the trust model is broken. Low

Gating clause under the agreed assumptions. Router holds no balance between txs; requires pre-stranded funds

Status

Acknowledged

[L-11] PTRW resolvePtrw one-shot

Severity rationale

Final verdict, Low. Add `if (ptrwResolved) revert`.

Description

`resolvePtrw` has no one-shot guard; an honest retry cannot overwrite (`:86` blocks it), but a tranched PT deposit with a resolve between tranches replaces the finalized ratio, and 1 wei of donated expired PT erases a real 20% depeg.

Impact

Impact ceiling if the trust model is broken. High if it lands

Gating clause under the agreed assumptions. Requires a second post-maturity PT arrival plus a second operator call (assumption 2)

Recommendation

Add `if (ptrwResolved) revert`

Status

Acknowledged

QA

[Q-01] TapirVrpOracle - VRP/PTRW resolution lacks a re-resolution guard

Severity rationale

Final verdict, Informational. Original scan severity, High. Status against the current source, VRP path fixed; PTRW path open; manual VRP path widened.

Description

The VRP path is fixed. `resolveVrp()` now reverts `VrpAlreadyResolved` on a second call (`TapirVrpOracle.sol:88`), closing the zero-preview re-sample that manufactured a max depeg.

The residual is `manualBackupResolveVrp()` (`:107-113`): no resolved-check, no bound, no timelock, and it emits the same `VrpResolved` event as the honest path. A call before `resolveVrp()` permanently blocks the honest path. A call after `writePriceData()` flips the pool from 0 bp to 9999 bp of depeg. Every harmful use requires `DEFAULT_ADMIN_ROLE` to act against users, excluded by assumption 1, so it caps at Informational. Add a defensive `if (vrpResolved) revert` for the pre-emption and liveness angle, which is a mistake-not-malice hazard.

Status

Acknowledged

[Q-02] Constructor and factory omit duration / minPrice bound checks

Severity rationale

Final verdict, Informational (missing basic validation). Original scan severity, Medium (detail says Low). Status against the current source, Open.

Description

A direct (non-factory) deploy with `minPrice = 0` lets the oracle write `hwm = 0`; `getState()` then reads `resolutionPrice == 0 && hwmPrice == 0` as unset and the pool sticks in COOLDOWN forever with collateral locked. `poolActiveDuration = 0` also passes the factory, and the repair path (`setCooldownDuration`) is ACTIVE-only, so it reverts too. The constructor additionally accepts `MIN_PRICE > MAX_PRICE`.

This is the missing-basic-validation cap, and assumption 4 places correct initialisation on the trusted deployer. Defence in depth: replicate the factory's bound checks in the constructor and bound both durations.

Status

Resolved

[Q-03] TapirOracle - Oracle setConfig/setSources are instant and unvalidated

Severity rationale

Final verdict, Informational. Original scan severity, High (detail says Low). Status against the current source, Open.

Description

Five distinct sub-cases, each reachable in one transaction. `minValidSources = 0` reverts every checkpoint; a lookback larger than `block.timestamp` underflows to `Panic(0x11)` at `TapirOracle.sol:380`; `xChainMode = true` with a zero messenger is accepted and then reverts at write; `depegPool` is repointable instantly; `setSources` can point all four "independent" feeds at one contract. The claim "minValidSources > 4 bricks checkpointing permanently" does not hold, one further `setConfig(4)` restores it.

The asymmetry with the pool's 7-day oracle timelock is real, but every path requires the trusted oracle admin to misconfigure, excluded by assumption 2 and by the missing-validation and admin-error caps. Recommendation: validate ranges in the setters (`1 ≤ minValidSources ≤ 4`, sane lookback, messenger set when `xChainMode`), and consider making `depegPool` write-once.

Status

Resolved

[Q-04] TapirPtrwOracle - PTRW depeg factor mixes token decimals

Severity rationale

Final verdict, Informational. Original scan severity, Medium. Status against the current source, Open.

Description

`ptrwErv` is in PT decimals and `ptrwArv` in base decimals; the ratio is dimensionless only when they match, and the constructor validates addresses, never decimals. With PT at 18 dp and base at 6 dp at an economically par redemption, `ptrwDepeg` computes to 0, so the closing price is 0; a real 50% depeg produces the identical 0, so the signal carries no information. Correction to the original write-up: the "absurd over-large value" direction is impossible, because `TapirPtrwOracle.sol:171-173` clamps to `PTRW_ACCURACY`. The reverse mismatch is a silent no-op that ignores a real depeg, arguably worse than a revert, not an inflation.

Assumption 4 guarantees correct decimals, and natural configurations (redeeming to the PT's own underlying) keep decimals equal. This is the admin-error (wrong parameters) cap. Recommendation: read and reconcile `decimals()` in the constructor, or store an explicit scale factor.

Status

Acknowledged

[Q-05] DepegPool - Redemption fee changeable at any time, including mid-REDEMPTIONS

Severity rationale

Final verdict, Informational (Low mechanic, capped). Original scan severity, Info. Status against the current source, Open.

Description

`setRedemptionFeeBp` (`DepegPool.sol:287-291`) has no state gate and is callable by admin or operator. A fee raised from 0 to the 255 bp maximum in the same block as a user redemption charges that user 2.55% instead of 0.

Bounded by the `uint8` cap at 2.55%, and gated to a trusted admin or operator under assumption 1 (the malicious-admin cap applies to the hostile-timing framing). Defence in depth: freeze the fee once the pool enters RESOLUTION/REDEMPTIONS, or emit-and-delay changes.

Status

Acknowledged

[Q-06] DepegPool - Core collateral becomes rescuable on schedule, not on actual resolution

Severity rationale

Final verdict, Informational. Original scan severity, Medium. Status against the current source, Open.

Description

`rescueErc20` for core tokens (`DepegPool.sol:338-353`) unlocks at `startTime + poolActiveDuration + cooldownDuration + 30 days` with no `depegResolved` requirement, no REDEMPTIONS gate, and no outstanding-claim coverage check. An admin can sweep the full backing to the treasury with DP and YB still outstanding in REDEMPTIONS, after which user redemption reverts. `sweepFeesToTreasury` does reserve claims; `rescueErc20` does not.

Both admin and operator are trusted (assumption 1), and the team accepts this as a v1 safety valve. The raw mechanic is total-loss authority, so the recommendation stands: gate core-collateral recovery on an actual resolution timestamp plus a claim period, restrict it to multisig, and retain coverage for outstanding supply. The related pause-clock asymmetry is covered in section 4.

Status

Acknowledged

[Q-07] Independent clamp

Severity rationale

Final verdict, Informational, with a caveat: `initialVrpPreview` is fixed at oracle construction, so headroom compounds across every pool the oracle serves (a 2× ceiling is consumed in ~7.3 years at 10% APY). Require a modelled headroom budget; the clamp turned a loud revert into a silent wrong settlement.

Description

`writePriceData` clamps adjusted HWM and closing to `MAX_PRICE` independently (`:152-157`); the pool derives depeg from their ratio, so clamping distorts it. HWM clamped alone reports 1000 bp for a true 2800 bp loss; both clamped reports 0 bp for a true 1600 bp loss.

Impact

Impact ceiling if the trust model is broken. High (silent mis-settlement)

Gating clause under the agreed assumptions. Assumption 4 (sufficient headroom) plus second-order-effect-from-a-fix

Recommendation

with a caveat: `initialVrpPreview` is fixed at oracle construction, so headroom compounds across every pool the oracle serves (a 2× ceiling is consumed in ~7.3 years at 10% APY). Require a modelled headroom budget; the clamp turned a loud revert into a silent wrong settlement

Status

Acknowledged

[Q-08] DepegPool - Sustained-pause rescue

Severity rationale

Final verdict, Informational, but note the role asymmetry: `pause()` is `onlyAdminOrOperator` while `unpause()` is admin-only, so a lone operator can start and hold the clock. Worth a defensive fix even under the trust model.

Description

Holding a pause open 30 days satisfies the early-rescue branch (`DepegPool.sol:344-350`) while `whenNotPaused` freezes user exits, so the full backing can be swept to the treasury while `paused == true`, days before the resolution unlock. The blip variant is fixed.

Impact

Impact ceiling if the trust model is broken. High (fund lock/sweep)

Gating clause under the agreed assumptions. Assumption 1 (admin/operator honest)

Recommendation

but note the role asymmetry: `pause()` is `onlyAdminOrOperator` while `unpause()` is admin-only, so a lone operator can start and hold the clock. Worth a defensive fix even under the trust model

Status

Acknowledged

[Q-09] TapirOracle - Oracle `pause()` scope

Severity rationale

Final verdict, Informational.

Description

`whenNotPaused` is only on `checkpoint()` (`TapirOracle.sol:257`); while paused, `record*Price` and `writePriceData` still run, so the pool can settle while the oracle is "paused".

Impact

Impact ceiling if the trust model is broken. Low

Gating clause under the agreed assumptions. Design; matches its own natspec

Status

Resolved

[Q-10] TapirOracle#constructor - Constructor accepts admin set to the zero address

Severity rationale

Final verdict, Informational. Low-value observation noted during review.

Description

The oracle constructor accepts `admin == address(0)` (`TapirOracle.sol:106`), which yields a permanently inert instance.

Status

Resolved

[Q-11] TapirOracle - Configured source decimals are never reconciled with the feed

Severity rationale

Final verdict, Informational. Low-value observation noted during review.

Description

The configured source `decimals()` is declared but never reconciled with the feed's own decimals (`TapirOracle.sol:60`).

Status

Acknowledged

[Q-12] TapirOracle#writePriceData - ETH sent to same-chain writePriceData is unrecoverable

Severity rationale

Final verdict, Informational. Low-value observation noted during review.

Description

The function is `payable`, `msg.value` is ignored, and there is no `receive` or withdraw path, so ETH sent to a same-chain call cannot be recovered.

Status

Acknowledged

[Q-13] TapirOracle - Two-source bootstrap averages the first checkpoint and resolves ties to the later slot

Severity rationale

Final verdict, Informational. Low-value observation noted during review.

Description

The two-source bootstrap averages the first checkpoint and resolves ties to the later slot.

Status

Acknowledged

[Q-14] TapirPtrwOracle - PTRW resolution deadlocks with non-standard base tokens

Severity rationale

Out of scope under the engagement brief, so it is not a live finding. It is recorded here for completeness and traceability against the submitted list. Original scan severity, Medium. Status against the current source, Open (still raw `IERC20.approve`/`transfer` at `TapirPtrwOracle.sol:97,124`).

Description

With a USDT-style base token that returns no data, `resolvePtrw()` reverts after the PT has been redeemed. The revert restores the PT balance, so `forceManualResolve` is never persisted, and `manualBackupResolvePtrw()` is then blocked by its `PtBalanceNotZero` check. There is no sweep, rescue, or withdraw function on the oracle, so the PT is permanently stranded.

The engagement's Integration Boundaries state base assets follow standard `IERC20`, and the contract restates the precondition in code (`TapirPtrwOracle.sol:48`, "Must be standard ERC20"). Out-of-scope findings are invalid; even if judged in scope, the weird-ERC20 cap forces Low. The one weakness that survives regardless of token type, the absence of any PT exit path, is recorded in section 4. Recommendation: adopt `SafeERC20` and add a deployment-time allowlist. A natspec warning is weaker than an enforced gate.

Status

Acknowledged

[Q-15] DepegToken#burn - No allowance check

Severity rationale

Out of scope under the engagement brief, so it is not a live finding. It is recorded here for completeness and traceability against the submitted list. Original scan severity, Low (detail says Info). Status against the current source, Root cause out of scope; shipped router safe.

Description

`DepegToken.sol` is excluded from scope. The mechanic is real: an authorised router can burn or redeem a third party's DP/YB with zero allowance. But the shipped `TapirRouter` hardcodes `msg.sender` as counterparty, so it is unreachable, and any future router is covered by assumption 1. Side-observations (Info): the router's DP/YB approvals to the pool (`TapirRouter.sol:145-146`) are dead code because the pool burns directly, and the natspec claiming users must approve DP/YB is inaccurate.

Status

Acknowledged

[Q-16] Nominal split accounting breaks for fee-on-transfer / rebasing assets

Severity rationale

Out of scope under the engagement brief, so it is not a live finding. It is recorded here for completeness and traceability against the submitted list. Original scan severity, Medium. Status against the current source, Open.

Description

With a 10% fee-on-transfer token, a deposit mints claims on the requested amount while the pool receives only the post-fee amount; the first exiter withdraws the nominal amount and a later holder is left short and reverts. Out of scope: the Integration Boundaries name fee-on-transfer and rebasing tokens explicitly. Worth a documented asset-listing criterion, but not a severity finding under the engagement.

Status

Acknowledged

[Q-17] Repository hygiene, hardcoded RPC key, duplicate config key

Severity rationale

Out of scope under the engagement brief, so it is not a live finding. It is recorded here for completeness and traceability against the submitted list. Original scan severity, Low (detail Info). Status against the current source, Open.

Description

The Hardhat config embeds a live RPC key, and the in-process network key is defined twice, the second silently overriding the first and forcing a public-RPC fork. None of this sits in the in-scope contract set. Operationally: rotate the key (it is in git history), move it to env-only, and de-duplicate the network config. No on-chain impact.

Status

Acknowledged

[Q-18] UniswapV3Math - Carries an unresolved production TODO

Severity rationale

Out of scope under the engagement brief, so it is not a live finding. It is recorded here for completeness and traceability against the submitted list. Original scan severity, Info. Status against the current source, Unchanged.

Description

The TODO banner remains in `contracts/libraries/UniswapV3Math.sol`, but `libraries/` (except `TimeDecay.sol`) is excluded from scope, and both `UniswapV3Math` and `TimeDecay` are referenced only from test-support mocks, not from any deployed contract. Resolve or delete before a future deployment wires the modified math in.

Status

Acknowledged

[Q-19] Deployment workflow relies on private-key env vars and deployer-admin defaults

Severity rationale

Out of scope under the engagement brief, so it is not a live finding. It is recorded here for completeness and traceability against the submitted list. Original scan severity, Info. Status against the current source, Open.

Description

Deployment scripts and build configuration sit outside the in-scope contract set. The advice is sound (encrypted keystores, multisig admin, fail-closed on unset production roles), but it is not a contract finding.

Status

Acknowledged

[Q-20] VRP trusts manipulable previewRedeem quotes

Severity rationale

Invalid under the engagement assumptions. It is recorded here for completeness and traceability against the submitted list. Original scan severity, Info. Status against the current source, Open.

Description

The VRP oracle stores `previewRedeem(witnessShares)` and treats it as realizable value. Donations, temporary liquidity constraints, withdrawal fees, and strategy accounting can desync the preview from actual proceeds, but the VRP assumption excludes preview manipulation and unreliability. No manipulation-free exploitation path remains. The one surviving non-manipulation concern, a vault-side revert that halts the checkpoint pipeline, is a distinct issue recorded in section 4, not this finding.

Status

Acknowledged

Centralisation

The Tapir Money project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Tapir Money project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au